

Informe Exhaustivo de Depuración - Proyecto DINASTÍA

1. Resumen Ejecutivo

Este informe detalla el análisis técnico y visual de los errores reportados en la capa de presentación (Visor) del sistema de partidos **DINASTÍA**, desarrollado en Godot Engine. A pesar de los diagnósticos iniciales generados por herramientas automatizadas que descartaban errores lógicos en la estructura ósea, la revisión manual de las capturas y secuencias de video (incluyendo la aparición de modelos desproporcionados y deformaciones de cámara) indica que el problema no radica en el motor matemático del partido (nucleo/), sino en la proyección espacial, escala de instancias y configuración de cámara en la interfaz 3D (visor/ y ui/).

2. Análisis del Motor y Lógica (Capa Núcleo vs Visor)

El Contrato Arquitectónico

El sistema está bien diseñado con una clara separación de responsabilidades (MVC/Arquitectura de 3 capas). El archivo nucleo/partido.gd maneja la verdad inmutable (matemáticas, reglas, RNG). El visor (MatchPlayback.gd) se limita a reaccionar. Dado que las mecánicas puras (goles, fallos, posiciones tácticas base) funcionan a nivel de consola, podemos aislar la depuración puramente a la interpretación espacial y la renderización en Godot.

El Problema de la Sincronización Visual

El error surge en la transición de datos del diccionario de sucesos a transformaciones 3D reales. Aunque el código de movimiento calcula un *objetivo* correcto (interpolando posiciones basadas en táctica, separación e inercia), la representación final falla porque el entorno que aloja estos cálculos tiene discrepancias de escala global.

3. Análisis de Anomalías Visuales (Basado en Evidencia)

Anomalía A: Problemas de Escala e Instanciación Fantasma (El "Gigante")

El error más crítico reportado visualmente no es un problema de rotación ósea (Spine o Neck). Se trata de un modelo 3D instanciado con escala incorrecta en el centro del mundo (0, 0, 0).

- **Causa Probable:** Un nodo de tipo MeshInstance3D o CharacterBody3D dejado por accidente en la escena principal estadio.tscn durante pruebas de desarrollo, o instanciado por defecto sin asignar la escala correcta requerida por el modo Manager.

- **Impacto:** Rompe la inmersión por completo y fuerza a la cámara a hacer clipping (atravesar la malla) cuando pasa por el origen.

Anomalía B: Deformación Ortográfica y Postura "Aplastada"

Los jugadores se ven extremadamente finos o "aplastados" contra el césped en la vista cenital. Claude diagnosticó ángulos óseos de -13.8° , pero el problema real es de proyección matemática de la cámara, no del esqueleto.

- **Causa Probable:** La Camera3D del modo Manager (VistaEstadio) está configurada en modo **Orthogonal** o tiene un modo **Perspective** con un FOV altísimo y mal ángulo. Esto aplasta la geometría de los modelos 3D al rasterizarlos en pantalla.

Anomalía C: Offset de Elementos de Entorno (Banquillos)

Como se observa en la captura proporcionada, aunque el bug de coordenadas invertidas X/Z se solucionó y ya no están en el área chica, las bancas se ven como bloques negros masivos y desproporcionados junto a la línea de banda, interrumpiendo el flujo visual de la cancha.

4. Plan de Acción y Soluciones Técnicas (Godot 4)

Área de Falla	Solución en Godot	Archivo / Nodo a Editar
Jugador Fantasma (Gigante)	Abrir el árbol de escena y buscar modelos 3D hijos en la raíz del campo. Eliminar todo nodo de jugador que no sea instanciado dinámicamente por MatchPlayback. Revisar función <code>_ready()</code> para evitar cargar assets de prueba ocultos.	estadio.tscn / estadio.gd
Cámara y Deformación (Postura)	Cambiar el nodo Camera3D: Si está en Orthogonal, pasar a Perspective. Configurar el FOV (Field of View) a un valor muy bajo (15-20 grados) y alejar la cámara drásticamente en el eje Y. Esto crea un "falso ortográfico" (isometric) que no aplasta los modelos 3D y mantiene el volumen natural sin verse encorvados.	ui/estadio.gd (Configuración de Nodo Camera3D)
Banquillos Sobredimensionados	En el script de construcción procedural, multiplicar la escala base de la malla de los banquillos por un factor ajustado (ej. 0.2), o alinear su posición relativa calculando los	stadium_builder.gd

Área de Falla	Solución en Godot	Archivo / Nodo a Editar
	bordes con AABB.	
Transición de Animaciones (Deslizamiento)	Configurar en el AnimationTree de los jugadores que el blend de la animación de "run" se controle directamente por el parámetro velocity.length() derivado de la inercia calculada en _mover(), asegurando que los pies coincidan con la velocidad real dictada por la táctica.	MatchPlayback.gd / AnimationTree

5. Herramientas Gratuitas de Asistencia con IA para Godot

El problema con inteligencias artificiales de terminal pura es que son "ciegas" al entorno espacial. Intentan justificar visuales rotos manipulando matrices en código sin ver la escena. Se recomiendan estas integraciones para Godot:

1. **AI Copilot (Plugin Open Source):** Plugin 100% gratuito instalable desde la AssetLib de Godot. Se ancla al editor, lee tu Scene Tree real, detecta errores de ejecución (runtime) y entiende de qué nodos le estás hablando.
2. **Cursor (con MCP Bridge de Godot):** Cursor IDE (que tiene un modo gratuito muy sólido) combinado con un servidor MCP de Godot. Esto le da "ojos" a la IA sobre tus archivos .tscn y no solo el código.
3. **GameDev Assistant:** Otro plugin in-editor con un nivel gratuito excelente, ideal para consultas rápidas sin tener que salir del motor.